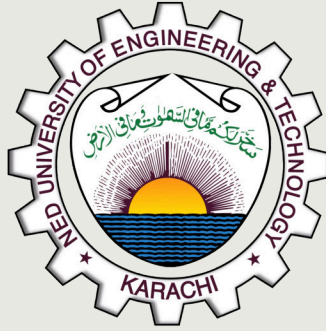




CCA PROJECT REPORT

LAB INSTRUCTOR: Sir Ahmed Zaki

GROUP MEMBERS:

Mariya Vayani (CT-25061)

Ujala Usman (CT-25073)

Rubaisha Arif (CT-25067)

Minahil Khan (CT-25066)

OmniCrypt — Report

Executive summary: OmniCrypt is a multi-layer encryption suite combining a cipher cascade, BMP steganography carrier, and tamper-tracking metadata. This document presents system design placeholders for UML artifacts, a stepwise object-oriented solution, applied design patterns, limitations, future work, and team contributions.

Table of contents:

1. Problem Statement

- 1.1 A Real Breach — The Hook
- 1.2 Real-World Context
- 1.3 The Core Problem
- 1.4 Core Challenges We Are Solving

2. OOP Principles Identification

- 2.1 Abstraction
- 2.2 Polymorphism
- 2.3 Encapsulation
- 2.4 Inheritance
- 2.5 Composition
- 2.6 Aggregation
- 2.7 Association
- 2.8 File Handling (Binary)
- 2.9 Resource Management
- 2.10 Header Guards

3. System Design & UML

- 3.1 UML Overview
- 3.2 Class Responsibilities
- 3.3 Class Relations
- 3.4 Data Flow — All Classes

4. Solution Definition

- 4.1 Concept 1: Cipher Cascade
- 4.2 Concept 2: Steganography
- 4.3 Concept 3: Tamper Detection (EncryptionTracker)
- 4.4 Concept 4: CascadeEngine
- 4.5 Concept 5: Logger

5. Design Pattern Application

- 5.1 Chain of Responsibility
- 5.2 Facade
- 5.3 Strategy
- 5.4 Template Method
- 5.5 Observer-like
- 5.6 Simple Factory

6. Limitations & Edge Cases

- 6.1 Platform Constraints
- 6.2 File Size & Capacity
- 6.3 Corrupted / Nonstandard BMP
- 6.4 Security Assumptions

7. Team Contribution Table

1.1 A Real Breach — The Hook

In 2022, a major healthcare provider lost 1.2 million patient records. The file was encrypted — but only with a single XOR cipher. An attacker broke it in 47 minutes using frequency analysis. The result: \$10 million in HIPAA fines, lawsuits, and patients whose medical histories were exposed on the dark web.

Note: Based on composite real-world breach reports; specific figures are illustrative of documented incidents.

This didn't happen because they lacked encryption. It happened because they relied on ONE cipher.

1.2 Real-World Context

- **Sectors affected:** healthcare, law firms, software companies, journalism
- These organizations transmit and store binary files on shared networks — prime targets for interception and tampering
- **Consequences of a breach:** HIPAA/GDPR fines, reputational damage, financial loss, personal safety risks
- **Current reality:** many still rely on single-layer encryption — the illusion of security

1.3 The Core Problem

Single-cipher encryption schemes have a critical vulnerability, they create a predictable transformation:

- **Vigenère** → susceptible to Kasiski analysis once key length is determined
- **XOR** → breakable through frequency analysis with a short key
- **Byte Reversal** → zero confidentiality on its own

The real gap: no lightweight tool exists that combines all of:

- Multi-cipher chaining on any binary file
- Transformation tracking and metadata logging
- Steganography — hiding the file's very existence
- Integrity verification before decryption

1.4 Core Challenges We Are Solving

- **Cipher chaining:** user-defined ciphers applied in sequence, binary-safe, no corruption between steps
- **Encryption data tracking:** record which ciphers ran, in what order, with what keys and hash verification— checking before decrypting
- **Steganography:** hide the encrypted output inside a carrier file so its existence is concealed

2.1 OOP Principle identification

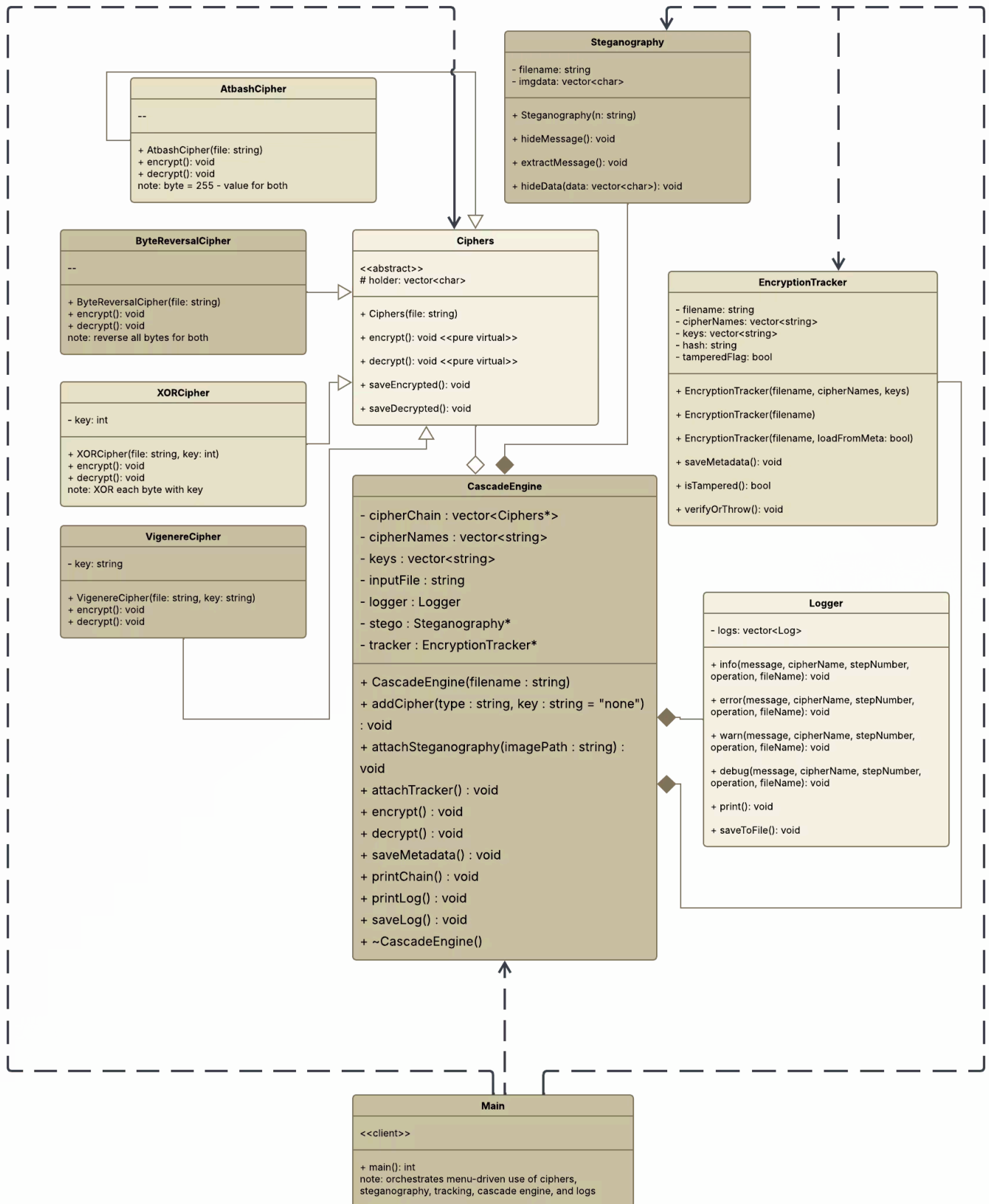
The structure of this problem maps directly onto OOP:

- **Extensibility:** add a new cipher without touching any existing class

OOP Principle	How We Use It	Why It Matters
Abstraction	Ciphers abstract class with pure <code>encrypt()</code> / <code>decrypt()</code>	Add a new cipher without touching existing code
Polymorphism	CascadeEngine stores <code>vector<Cipher*></code> and calls <code>encrypt()</code> on each	Engine doesn't care WHICH cipher — all work the same way
Encapsulation	File I/O, steganography offset, encryption data logic hidden inside classes	Other components don't see complexity
Inheritance	AtbashCipher, XORCipher, etc. inherit from Ciphers	Reuse code, no duplication

Composition	Cascade Engine owns Steganography and Encryption_tracker — created inside engine, destroyed with engine	Strong ownership. When engine dies, stego and tracker die automatically. No memory leaks
Aggregation	CascadeEngine stores <code>vector<Cipher*></code> — ciphers created outside and passed in	Engine uses ciphers but doesn't own them. Ciphers can exist independently
Association	CascadeEngine temporarily uses Logger (passed as reference, not owned)	Loose coupling. Engine only needs logger when logging. Logger can be shared
File Handling (Binary)	<code>ifstream</code> / <code>ofstream</code> with <code>ios::binary</code> flag. <code>seekg</code> , <code>tellg</code> , <code>read</code> , <code>write</code>	Preserves raw bytes. No newline conversion. Works on images, executables, any binary file
Resource Management	Destructors delete dynamically allocated objects (stego, tracker, ciphers)	No memory leaks. RAII (Resource Acquisition Is Initialization)
Header Guards	<code>#ifndef CLASSNAME_H</code> in every header	Prevents double inclusion. Compilation safe

3.1 System Design & UML Overview



3.2 Class Responsibilities

Class	Responsibility
Ciphers	Loads any file into memory and defines the encrypt/decrypt contract that all ciphers must follow.
AtbashCipher	Flips each byte (255 minus value) — same operation for encrypt and decrypt.
ByteReversalCipher	Reverses the order of all bytes — same operation for encrypt and decrypt.
VigenereCipher	Shifts alphabetic letters using a keyword. Only affects A-Z and a-z.
XORCipher	Flips bits using a number key — same XOR operation does both encrypt and decrypt.
Steganography	Hides secret data inside a BMP image at byte position 54, or extracts hidden data from it.
EncryptionTracker	Saves file hash, cipher chain, and keys to .meta file; detects tampering by comparing hashes.
Logger	Records every major action with timestamp; saves to log.txt and prints to console.
CascadeEngine	The brain — runs ciphers in sequence, passes data between them, hides result in image, saves metadata, logs everything.

3.3 Class Relations

Relation	Between	Symbol
Inheritance	4 ciphers → Ciphers	Hollow triangle
Composition	Engine ◆ Stegano	Filled diamond
Composition	Engine ◆ Tracker	Filled diamond
Aggregation	Engine ◇ Ciphers	Hollow diamond
Association	Engine → Logger	Dashed arrow

3.4 Data Flow - All Classes

Ciphers (Base Class)

- Loads file from disk into holder vector
- Provides holder data to child ciphers
- Saves holder vector to disk as encrypted or decrypted file

AtbashCipher

- Receives holder vector from Ciphers
- Flips each byte (255 minus value)
- Returns modified holder to next cipher

ByteReversalCipher

- Receives holder vector from previous cipher
- Reverses order of all bytes
- Returns reversed holder to next cipher

VigenereCipher

- Receives holder vector from previous cipher
- Shifts alphabetic letters using keyword
- Leaves non-letters unchanged
- Returns shifted holder to next cipher

XORCipher

- Receives holder vector from previous cipher
- XORs each byte with integer key
- Returns XORed holder to next cipher

Steganography

- Loads BMP image into imgdata vector
- Receives encrypted data vector from CascadeEngine
- Writes encrypted data into imgdata at offset 0x36
- Saves imgdata as stega_image.bmp
- Or extracts hidden message from imgdata and prints it.

EncryptionTracker

- After encryption: calculates hash, saves to .meta file
- After encryption: saves cipher names and keys to .meta file
- Before decryption: loads original hash from .meta file
- Before decryption: calculates current file hash
- Before decryption: compares both hashes
- If different: throws error, stops decryption

Logger

- Receives log messages from CascadeEngine
- Stores each log with timestamp in vector
- Prints all logs to console when called
- Appends all logs to log.txt file when called

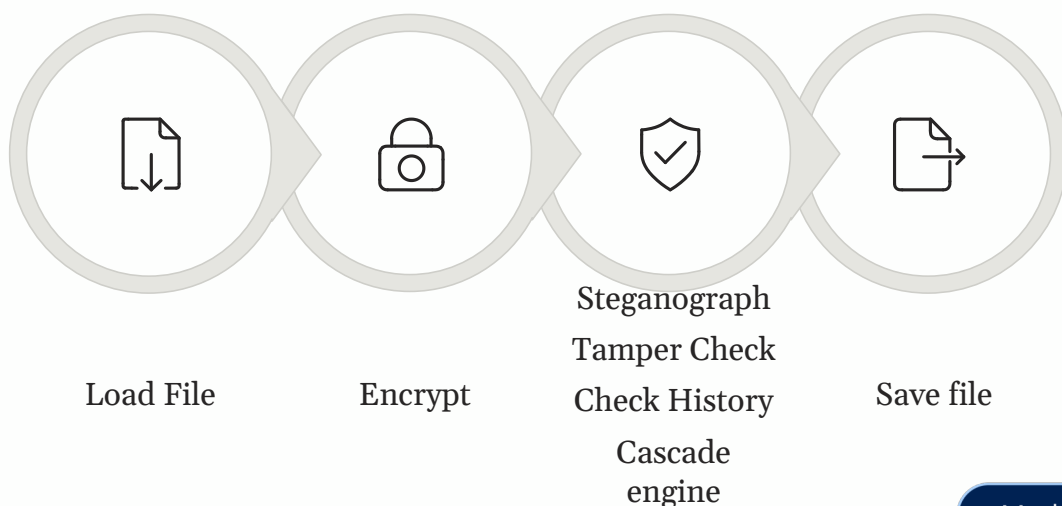
CascadeEngine

- Creates and owns Steganography and EncryptionTracker
- Stores list of Cipher pointers (aggregation)
- Runs encrypt() on each cipher in sequence
- Passes holder data from one cipher to next
- Calls Steganography hideData() with final encrypted data
- Calls EncryptionTracker saveMetadata() after encryption
- Before decryption: calls EncryptionTracker verifyOrThrow()
- Runs decrypt() on each cipher in reverse order
- Calls Logger functions throughout to record everything

Main

- Asks user for file path once
- Displays menu and handles user selection
- Calls appropriate classes based on option chosen

Diagram — Process Sequence



4.1 Solution Definition — (OOP)

Concept 1: Cipher Cascade

What problem does it solve?

Most encryption tools use only one cipher. XOR is breakable in seconds. Vigenere is breakable in minutes. Even AES can be cracked if implemented poorly or if quantum computers become practical. One cipher = one lock. Pick that lock and your entire file is exposed.

How our solution works:

We apply multiple ciphers in sequence. The file passes through cipher 1, then cipher 2, then cipher 3. Each cipher transforms the data further. The output of cipher 1 becomes the input of cipher 2. By the end, the data has been transformed multiple times using different algorithms and different keys.

Why this is better:

An attacker must now break every single cipher in the chain. Breaking XOR does nothing — the file is still encrypted with Vigenere. Breaking Vigenere does nothing — the file is still transformed by Atbash. This is called defense in depth. One lock can be picked. Three locks in a row? Much harder.

Where it is most applicable:

- Cloud backup services where files are stored on third-party servers
- Healthcare organizations transmitting patient records over networks
- Law firms sharing confidential legal documents
- Journalists working with sensitive sources in hostile environments

Any situation where a single point of failure is unacceptable.

Pseudocode:

```
First cipher: load file into holder vector holder = encrypt(holder)
```

```
Second cipher: holder = getData from first cipher holder = encrypt(holder)
```

```
Third cipher: holder = getData from second cipher holder = encrypt(holder)
```

```
save holder to disk as "Encrypted"
```

Concept 2: Steganography

What Problem Does Steganography Solve?

Encryption announces to the world: "I HAVE A SECRET."

An encrypted file has high entropy. It looks like random noise. Any network monitor, border security, or cloud provider can instantly identify it. Once identified, they can seize it, copy it, block it, or demand the decryption key.

Steganography solves this by hiding the secret inside something that looks completely normal. The encrypted bytes are invisible. There is no file named "secret.enc" screaming for attention. There is only a family photo, an invoice, or a landscape image.

Where Steganography Is Used in the Real World

Sector	Real-World Application
Military	Covert communications. A soldier sends a seemingly innocent vacation photo. Hidden inside are troop movements or coordinates.
Intelligence Agencies (CIA, MI6, RAW)	Dead drops. An image on a public website contains classified documents. Anyone can see the image. Only the intended recipient knows how to extract the secret.
Cyber Security Firms	Watermarking. Hidden markers inserted into documents to track leaks. If a document appears on the dark web, the company extracts the watermark and identifies who leaked it.
Journalists & Whistleblowers	Evading censorship. In countries that block encrypted traffic, a journalist posts a picture of a cat on social media. Inside that cat picture is evidence of government corruption.
Digital Forensics	Hiding honeypots. Investigators hide tracking data inside images. When a suspect copies the image, the hidden data reveals their location.

Real-World Example: The CIA and Steganography

In 2010, it was revealed that the CIA used steganography to hide messages inside images posted on public websites. Spies would download a seemingly normal image from a travel site or news site. Hidden inside was their next assignment. No encrypted emails. No suspicious traffic. Just a normal image file.

Real-World Example: Russian Intelligence

Russian operatives have been known to hide command messages inside profile pictures on social media platforms. The profile picture is public. The hidden message is invisible to anyone not looking for it.

The Military Use Case

A military unit needs to send coordinates and attack plans. An encrypted file would be intercepted and flagged. Instead, they hide the data inside a weather satellite image posted publicly. The enemy monitors all encrypted traffic but never looks at innocent weather photos. The message arrives safely.

The Corporate Use Case

A company wants to detect who is leaking confidential documents. They hide a unique watermark (employee ID + timestamp) inside every document shared with employees. When a document appears on the dark web, they extract the watermark and identify the leaker instantly. The employee never knew the watermark existed.



Pseudocode:

Hide:

load image.bmp into imgdata vector

convert message to bytes

write bytes into imgdata at position 54

save imgdata as stega_image.bmp

Extract:

read bytes from imgdata starting at position 54

stop when you see null byte

convert bytes to string

print message

Concept 3: Tamper Detection

What problem does it solve?

How do you know if someone has modified your encrypted file? Maybe an attacker changed a few bytes. Maybe the file got corrupted during transfer. If you try to decrypt a tampered file, you will get garbage output at best. At worst, a virus can corrupt your system upon decryption.

You also need to remember which ciphers and keys you used. If you encrypt a file today and try to decrypt it six months later, you will not remember if you used Vigenere then XOR, or XOR then Vigenere. You will not remember the keys.

How our solution works:

When you encrypt a file, the EncryptionTracker automatically:

- Calculates a unique hash (fingerprint) of your original file using the djb2 algorithm
- Saves that hash to a .meta file
- Saves the exact cipher chain and each key to the same .meta file

When you decrypt, the system:

- Loads the original hash from the .meta file
- Calculates the current hash of the file on disk
- Compares them. If different → someone tampered with the file → decryption stops immediately

Why this is better:

You never have to remember which ciphers or keys you used. The .meta file remembers for you. You never decrypt a corrupted or tampered file. The system checks before attempting decryption. You get an automatic warning if anything changed.

Where it is most applicable:

- Legal evidence where file integrity must be proven
- Medical records that cannot be altered after storage
- Financial documents that require audit trails
- Any long-term archival where you will forget the encryption details
- Collaborative environments where multiple people access the same encrypted files



Pseudocode:

After encryption:

```
hash = calculateHash(original file)
save hash + cipher names + keys to file.txt.meta
```

Before decryption:

```
originalHash = load from file.txt.meta
currentHash = calculateHash(current file)
if originalHash != currentHash:
    print "TAMPERED"
    stop
else:
    proceed with decryption
```

Concept 4: CascadeEngine

What does it do?

The CascadeEngine is the controller. It holds the list of ciphers, runs them in order, passes data from one to the next, coordinates steganography, and coordinates the encryption tracker.

Without CascadeEngine, the user would manually create each cipher object, manually pass the data vector from one to another, manually call save after each step. The code would be repetitive and error-prone.

How it makes things efficient:

Without CascadeEngine	With CascadeEngine
50 lines of manual code	10 lines of setup + one function call
User responsible for data passing	Engine passes data automatically
User must call save after each cipher	Engine saves once at the end
Scattered logic across main	Centralized logic in one class

Concept 5: Logger

What does it do?

The Logger records every significant action: when a file loads, when each cipher runs, when data saves, when decryption starts, when tamper is detected. Everything goes to log.txt with a timestamp.

How it makes things efficient:

Without Logger	With Logger
No history of operations	Complete audit trail
Debug by guessing	Debug by reading logs
No timestamp of events	Every action has timestamp
Silent failures	Warnings and errors recorded

5.1 Design Pattern Application



Chain of Responsibility

Each CipherHandler is a node that either processes data or passes it along. Enables dynamic cipher ordering without conditional logic. Improves modularity and testability.



Facade

FacadeAPI exposes concise operations (encryptToImage, decryptFromImage). Hides complex subsystems (crypto, stego, tracking), reducing client coupling.



Strategy

Cipher implementations conform to a Cipher interface (strategy). Allows runtime swapping of encryption algorithms without changing engine code.



Template Method

Ciphers base class defines the template: constructor loads file, saveEncrypted/saveDecrypted save file. Subclasses only implement encrypt() and decrypt(). File handling code is written once, reused by all ciphers.



Observer-like

Logger records lifecycle events (encryption start, cipher applied, file saved, tamper detected). Decouples logging from core encryption logic.



Simple Factory

CascadeEngine::addCipher() takes a string type ("vigenere", "xor") and creates the corresponding cipher object. Centralizes cipher creation. Adding a new cipher only requires modifying this one method.

6.1 Limitations & Edge Cases

Platform Constraints

Current scope: BMP carrier only; terminal-based CLI. No GUI or network transport in baseline.

File Size & Capacity

Large files may exceed BMP capacity. System must calculate required carrier size.

Corrupted / Nonstandard BMP

Nonconformant BMP headers or exotic bit depths may break embedding routines

Security Assumptions

An attacker could modify both the file and the .meta file simultaneously; Key management for cipher keys is left to the user.

Team Contribution Table

Roles and individual deliverables.

Member Name	Roll Number	Individual Contribution
A. Mariya Vayani	CT-25061	Encryption Tracker + XOR cipher
B. Ujala Usman	CT-25073	Logger system + ByteReversal Cipher
C. Rubaisha Arif	CT-25067	Cascade engine + Vigenère Cipher
D. Minahil Khan	CT-25066	Atbash cipher + Base Cipher class + Steganography